The `TokenTracker` class is a custom implementation to monitor and manage the token usage during interactions with the language model. This class tracks the tokens used in the current session but can also be extended to export these token metrics to an analytics service if needed.

The provided code creates an instance of `TokenTracker` to monitor the number of tokens used in the current session. When an API call to the language model completes, the `handleLLMEnd` callback function is triggered, extracting the message object from the output. It then updates the token tracker with usage metadata from the message, which includes details about input and output tokens.

> **NOTE** While this implementation captures token usage effectively through message metadata, there is an alternative method for counting tokens using `output.llmOutput?.tokenUsage`, but it is limited to certain models and may not be reliable across different providers. For example, since it uses js-tiktoken, it can only currently support counting tokens for OpenAI models and not Google Gemini models.

What can we do with this addition? For one, it allows us to gain precise insights into how many tokens are consumed during each API call, enabling better cost management and optimization of resources. Monitoring the token usage is crucial since it will allow us to refine our prompts to minimize unnecessary token usage.

Let's move on to the UI part of the application and showcase the first usable demo of the whole web application.

7.1.5 Demonstrating the app

As we develop the application to handle the goals we aim to achieve, we need to make some small adjustments to the UI we have been using so far to accommodate this new feature. These changes will improve the user experience and ensure that users can efficiently interact with the summarization functionality.

First, we will implement a single input field that allows users to either paste text directly or upload a document for summarization. While the user waits for the summarization to complete, we will include a message to keep them informed that their request is being processed. Finally, once the summary is generated, it will be displayed in a message format within the UI.

> **NOTE** The source code for the examples in this chapter is available at https://github.com/Generative-AI-Web-Apps/Code/tree/main/ch07/summarization-web-app.

Project code: summarization-web-app

The code for this example project is in the ch07/summarization-web-app folder. You can use that example by running this command in the root folder of the project repository: